



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIO DE CLASE 8

NOMBRE COMPLETO: Mora Ortega Judith

Nº de Cuenta: 318006538

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: Mayo/2026

CALIFICACIÓN: _____

Ejercicio de Clase

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

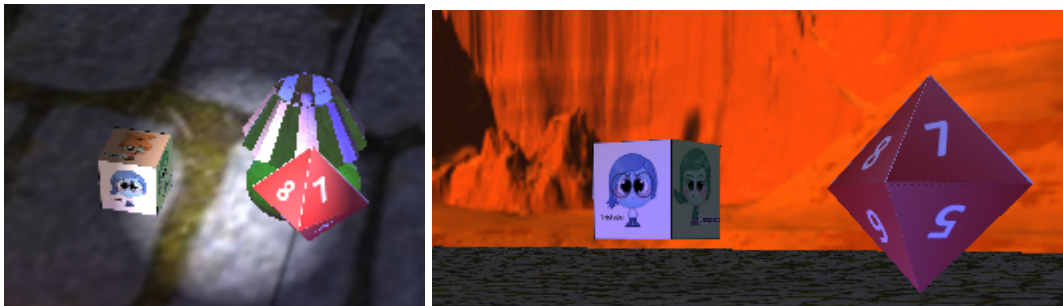
1.- Agregar su dado de 8 caras y editar sus normales para que las caras del dado sean iluminadas correctamente.

Para esto vamos a normalizar las caras 8 caras de octaedro ta texturizado, utilizando para un octaedro regular los vectores normales son ($\pm 1/\sqrt{3}$, $\pm 1/\sqrt{3}$, $\pm 1/\sqrt{3}$). $1/\sqrt{3} \approx 0.5774$ teniendo en cuenta este valor.

Cambios al Octaedro

```
GLfloat oct_vertices[] = {
    // Cara 1 - número "8"
    0.0f, 1.0f, 0.0f, 0.7540f, 0.0000f, 0.5774f, -0.5774f, -0.5774f,
    -1.0f, 0.0f, 0.0f, 0.5000f, 0.2483f, 0.5774f, -0.5774f, -0.5774f,
    0.0f, 0.0f, 1.0f, 1.0000f, 0.2483f, 0.5774f, -0.5774f, -0.5774f,
    // Cara 2 - número "6"
    0.0f, -1.0f, 0.0f, 1.0000f, 0.2483f, 0.5774f, 0.5774f, -0.5774f,
    0.0f, 0.0f, 1.0f, 0.7600f, 0.4965f, 0.5774f, 0.5774f, -0.5774f,
    -1.0f, 0.0f, 0.0f, 0.5000f, 0.2483f, 0.5774f, 0.5774f, -0.5774f,
    // Cara 3 - número "7"
    0.0f, 1.0f, 0.0f, 1.0000f, 0.7448f, -0.5774f, -0.5774f, -0.5774f,
    0.0f, 0.0f, 1.0f, 0.5000f, 0.7448f, -0.5774f, -0.5774f, -0.5774f,
    1.0f, 0.0f, 0.0f, 0.7600f, 0.4965f, -0.5774f, -0.5774f, -0.5774f,
    // Cara 4 - número "5"
    0.0f, -1.0f, 0.0f, 0.7520f, 0.9931f, -0.5774f, 0.5774f, -0.5774f,
    1.0f, 0.0f, 0.0f, 0.5000f, 0.7448f, -0.5774f, 0.5774f, -0.5774f,
    0.0f, 0.0f, 1.0f, 1.0000f, 0.7448f, -0.5774f, 0.5774f, -0.5774f,
    // Cara 5 - número "2"
    0.0f, 1.0f, 0.0f, 0.0000f, 0.2483f, -0.5774f, -0.5774f, 0.5774f,
    1.0f, 0.0f, 0.0f, 0.5000f, 0.2483f, -0.5774f, -0.5774f, 0.5774f,
    0.0f, 0.0f, -1.0f, 0.2400f, 0.4965f, -0.5774f, -0.5774f, 0.5774f,
    // Cara 6 - número "4"
    0.0f, -1.0f, 0.0f, 0.5000f, 0.2483f, -0.5774f, 0.5774f, 0.5774f,
    0.0f, 0.0f, -1.0f, 0.7600f, 0.4965f, -0.5774f, 0.5774f, 0.5774f,
    1.0f, 0.0f, 0.0f, 0.2460f, 0.4965f, -0.5774f, 0.5774f, 0.5774f,
    // Cara 7 - número "1"
    0.0f, 1.0f, 0.0f, 0.5000f, 0.7448f, 0.5774f, -0.5774f, 0.5774f,
    0.0f, 0.0f, -1.0f, 0.2400f, 0.4965f, 0.5774f, -0.5774f, 0.5774f,
    -1.0f, 0.0f, 0.0f, 0.7540f, 0.4965f, 0.5774f, -0.5774f, 0.5774f,
    // Cara 8 - número "3"
    0.0f, -1.0f, 0.0f, 0.0000f, 0.7448f, 0.5774f, 0.5774f, 0.5774f,
    -1.0f, 0.0f, 0.0f, 0.2400f, 0.4965f, 0.5774f, 0.5774f, 0.5774f,
    0.0f, 0.0f, -1.0f, 0.5000f, 0.7448f, 0.5774f, 0.5774f, 0.5774f,
};
unsigned int oct_indices[] = {
```

Ejecución del código. (Luces)



2.- Apagar con teclado la luz (pointlight) de su lámpara creada para el reporte de la práctica 7.

Para este punto se declara varias cosas, y que para activar el correcto funcionamiento de la misma ocuparemos la letra L

Para darle el funcionamiento de prender y apagar vamos a declarar un *Bool*

```
/// — Control de Lamp —  
bool lampOn = true;
```

La tecla que vamos a usar “L”

```
// — Input de la lámpara —  
void ProcessLampInput(bool* keys, GLfloat dt)  
{  
    static bool lPressed = false;  
    if (keys[GLFW_KEY_L] && !lPressed) {  
        lampOn = !lampOn;  
        lPressed = true;  
    }  
    if (!keys[GLFW_KEY_L]) lPressed = false;  
}
```

Creamos la iluminación el **spotLightCount[1]**

```

spotLightCount++;
pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
    0.0f, 1.0f,
    0.0f, -4.0f, -10.0f,
    0.3f, 0.2f, 0.1f);
pointLightCount++;

```

```

// Actualizar linterna a la posición de la cámara
glm::vec3 lowerLight = camera.getCameraPosition();
lowerLight.y -= 0.3f;
if (lampOn) {
    pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
        0.0f, 1.0f,
        0.0f, -4.0f, -10.0f,
        0.5f, 0.3f, 0.1f);    // ← más concentrada
}
else {
    pointLights[1] = PointLight(0.0f, 0.0f, 0.0f,
        0.0f, 0.0f,
        0.0f, -4.0f, -10.0f,
        0.5f, 0.3f, 0.1f);    // ← todo en 0, sin luz
}
unsigned int activeLights = lampOn ? pointLightCount : pointLightCount - 1;
spotLights[0].SetFlash(lowerLight, camera.getCameraDirection());

```

Esto nos sirve para prender y apagar la lámpara y que le manda la shader.

Ejecución



3.- Hacer que al presionar una tecla, la luz del faro del coche cicle de color entre rojo, amarillo, verde, cian, azul, magenta

Primero vamos a agregar el ciclo de colores.

```
// — Ciclo de color del faro del coche —
int faroColorIndex = 0;
bool faroKeyPressed = false;
glm::vec3 faroColores[] = {
    glm::vec3(1.0f, 0.0f, 0.0f),    // rojo
    glm::vec3(1.0f, 1.0f, 0.0f),    // amarillo
    glm::vec3(0.0f, 1.0f, 0.0f),    // verde
    glm::vec3(0.0f, 1.0f, 1.0f),    // cian
    glm::vec3(0.0f, 0.0f, 1.0f),    // azul
    glm::vec3(1.0f, 0.0f, 1.0f)     // magenta
};
```

Tecla que vamos ampliar es N

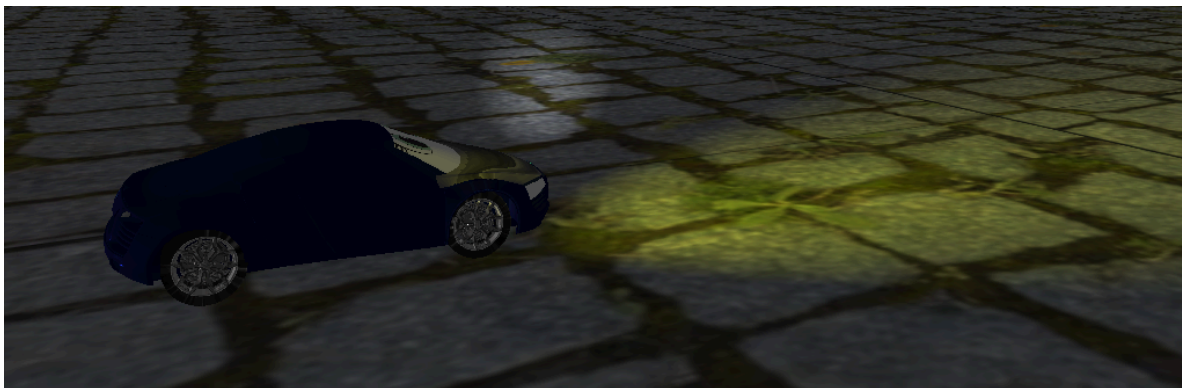
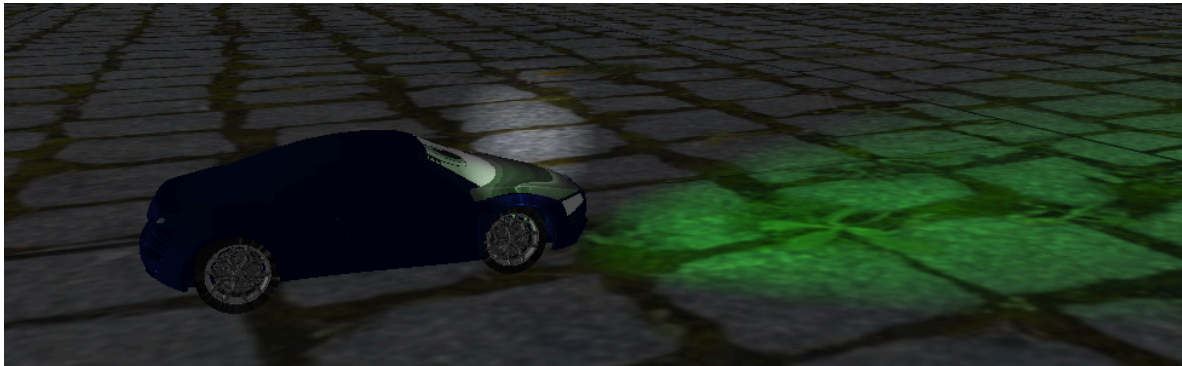
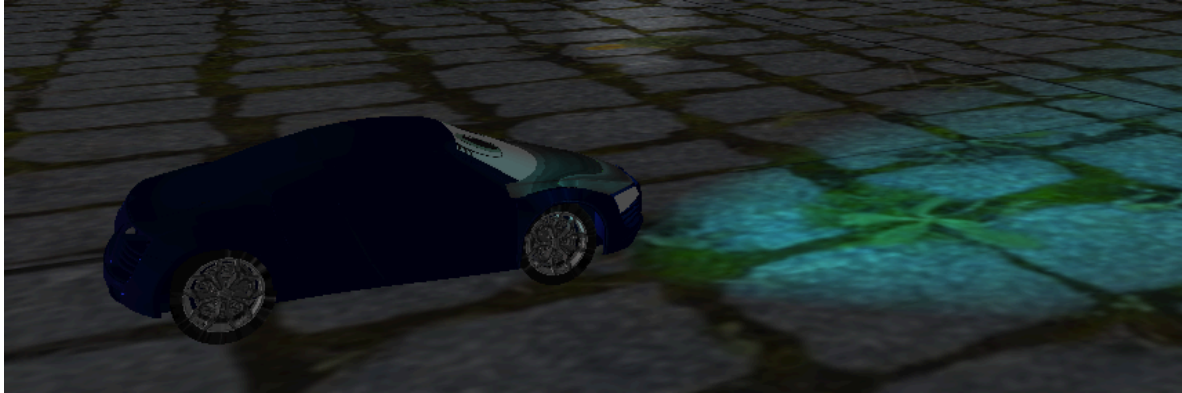
```
// — Input Faro de Colores con tecla N —
void ProcessFaroColorInput(bool* keys)
{
    if (keys[GLFW_KEY_N] && !faroKeyPressed)
    {
        faroColorIndex = (faroColorIndex + 1) % 6;
        faroKeyPressed = true;
    }
    if (!keys[GLFW_KEY_N])
        faroKeyPressed = false;
}
```

Para que esto siga al Auto en movimiento se cambió loop

```
// Faro frontal - se mueve con el coche
// Posición: enfrente del coche, a la altura del faro
glm::vec3 faroPos = carPosition + glm::vec3(2.0f, 1.5f, -1.5f);
glm::vec3 faroDir = glm::vec3(0.0f, -0.2f, -1.0f);
glm::vec3 c = faroColores[faroColorIndex];
spotLights[2] = SpotLight(c.r, c.g, c.b,
    0.0f, 2.0f,
    faroPos.x, faroPos.y, faroPos.z,
    faroDir.x, faroDir.y, faroDir.z,
    1.0f, 0.0f, 0.0f,
    25.0f);
```

Ejecución





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

1. La parte más difícil fue la normalización y mapeo el octaedro.
2. La parte de más cuenta es la importación de modelos y sus texturas, me tardo demasiado.
3. Fueron bastante tardadas.

3.- Conclusión

Que esta parte de los ejercicios en clase, me costó mucho la normalización de octaedro, para que pudiera reflejar correctamente las luces ocupadas, la parte de la lámpara ya la tenía solo era agregar un bool para prenda y se apague. Y la última parte fue el ciclo de colores, no fue tan difícil. Pero aun así me costó mucho trabajo hacer que vieran correctamente las texturas.

4.-Bibliografía en formato APA

Roque, J. (2026). *opengl3.3ydosmain* [Material de código]. UMAN.